Universidad Veracruzana
Facultad de Estadística e Informática · Región Xalapa
Lic. en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

Programación en Red — Unidad II · Programación a nivel de sockets

Manejo de errores y excepciones en programación de sockets

Teoría, códigos comentados y ejercicios de consolidación

Guía de clase · Martes 06 de octubre de 2026

Mtro. José Pablo Rueda Holguín · NRC 19141

1. Objetivos de la sesión

- Distinguir entre un error de sintaxis y una excepción en tiempo de ejecución.
- Manejar excepciones con try, except, else y finally de forma específica y ordenada.
- Reconocer las excepciones más frecuentes al programar con sockets TCP y UDP.
- Escribir clientes y servidores que no se detengan ante fallos de la red o del otro extremo.
- Consolidar lo aprendido sobre hilos, TCP y UDP mediante ejercicios progresivos.

2. Fundamento teórico: errores y excepciones

2.1 Error de sintaxis y excepción

Un **error de sintaxis** se detecta antes de ejecutar el programa: una instrucción mal escrita impide que el programa siquiera arranque. Una **excepción** es un error que ocurre durante la ejecución con un código sintácticamente correcto: dividir entre cero, convertir “hola” a entero o conectarse a un servidor apagado.

Cuando una excepción no se atiende, el programa **termina de forma abrupta** mostrando el traceback (la pila de llamadas hasta el punto del fallo). En un servidor de red esto es inaceptable: un cliente que se desconecta de golpe no debe tumbar a todo el servicio. El mecanismo para atender excepciones es la estructura try / except.

2.2 La estructura try / except / else / finally

- **try:** encierra el código que puede fallar.
- **except TipoDeError:** se ejecuta solo si ocurrió esa excepción; pueden existir varios except, uno por tipo de error, y Python usa el primero que coincide.
- **else:** se ejecuta solo si el bloque try no produjo ninguna excepción; separa el “camino feliz” del manejo de errores.

- **finally:** se ejecuta siempre, haya o no excepción, e incluso si hubo un return; es el lugar para liberar recursos: cerrar sockets, archivos o registrar el cierre.

Orden de evaluación: try → (except si hubo error | else si no hubo) → finally siempre. Capturar con except a secas, sin tipo, atrapa cualquier excepción y oculta errores de programación; debe evitarse.

2.3 Jerarquía de excepciones y códigos de error

Las excepciones forman una jerarquía de clases. Las relevantes para redes cuelgan de **OSError**, que representa errores del sistema operativo: cada una lleva un **código de error (errno)** numérico que identifica la causa exacta. Por ejemplo, “Address already in use” es errno 98 en Linux y “Connection refused” es errno 111. Capturar OSError atrapa todas sus subclases; capturar la subclase concreta (ConnectionRefusedError, TimeoutError…) permite responder a cada causa de forma distinta.

Jerarquía simplificada: Exception → OSError → ConnectionError → ConnectionRefusedError, ConnectionResetError, BrokenPipeError; y en paralelo OSError → TimeoutError. Por eso except OSError colocado antes que except ConnectionRefusedError dejaría sin efecto al segundo: **las clausulas except se escriben de lo más específico a lo más general**.

2.4 Excepciones típicas en programación de sockets

Excepción	Cuándo ocurre	Manejo recomendado
ConnectionRefusedError	connect() o create_connection() hacia un puerto donde no hay nadie escuchando	Avisar con claridad y terminar el cliente limpio, o reintentar tras una pausa
TimeoutError	recv() o recvfrom() no recibieron datos antes del tiempo de settimeout()	Reintentar un número limitado de veces; en UDP es la única forma de detectar pérdida
ConnectionResetError	El otro extremo cerró la conexión de forma abrupta mientras se leía	Dar por terminada esa conexión y seguir atendiendo a los demás clientes
BrokenPipeError	sendall() sobre una conexión que el otro extremo ya cerró	Cerrar el socket y liberar al hilo que atendía esa conexión
OSError: Address already in use	bind() sobre un puerto ocupado por otro proceso	Usar SO_REUSEADDR o elegir otro puerto; no reiniciar a ciegas
UnicodeDecodeError	decode("utf-8") sobre bytes que no respetan esa codificación	Indica que el otro extremo no sigue el protocolo acordado; registrar y descartar
KeyboardInterrupt	El usuario presiona Ctrl+C	Cerrar los sockets y terminar de forma ordenada

2.5 Reglas de un buen manejo de errores

- **Capturar excepciones específicas:** un except por cada causa que se sabe manejar; nunca except vacío.
- **No silenciar errores:** todo except debe dejar evidencia (mensaje en consola o registro); un error tragado en silencio se vuelve imposible de diagnosticar.
- **Liberar recursos en finally o con with:** los sockets se cierran siempre, falle o no lo que había en medio.
- **El servidor no muere por un cliente:** el try / except vive dentro de la función que atiende a cada cliente, de modo que el fallo de una conexión termine solo a su hilo.
- **Reintentos con límite:** en UDP toda consulta puede perderse; reintentar con un máximo y una espera creciente, nunca en un ciclo infinito.

3. Códigos comentados

Los cinco códigos siguientes progresan desde el try / except básico hasta clientes y servidores de red tolerantes a fallos. Cada uno va numerado línea a línea y seguido de su explicación.

Código 1 · primer_try.py

```
1  def dividir(a, b):
2      """Divide a entre b manejando los errores típicos."""
3      try:
4          resultado = a / b
5      except ZeroDivisionError:
6          print("Error: no se puede dividir entre cero")
7          return None
8      except TypeError:
9          print("Error: ambos valores deben ser numéricos")
10         return None
11     return resultado
12
13
14 print(dividir(10, 2))
15 print(dividir(10, 0))
16 print(dividir(10, "x"))
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
1–2	def dividir(a, b)	Función con docstring que divide dos valores manejando los errores típicos de la operación.
3–4	try:	Bloque vigilado: la división a / b puede lanzar ZeroDivisionError o TypeError.
5–7	except ZeroDivisionError	Si b vale cero, se informa y la función devuelve None en lugar de morir.

Línea(s)	Elemento	Explicación
8–10	except TypeError	Si algún valor no es numérico (por ejemplo, una cadena), se informa y se devuelve None.
11	return resultado	Si no hubo excepción, la función devuelve el cociente.
14–16	tres llamadas de prueba	Muestran los tres caminos: división normal, división entre cero y tipo incorrecto. El programa completa las tres sin detenerse.

Código 2 · try_completo.py

```python
def leer_entero(texto):
    """Convierte texto a entero mostrando else y finally."""
    try:
        valor = int(texto)
    except ValueError:
        print(f"'{texto}' no es un entero válido")
        return None
    else:
        print(f"Conversión exitosa: {valor}")
        return valor
    finally:
        print("Fin del intento de conversión")


leer_entero("42")
leer_entero("hola")
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
3–4	try: int(texto)	La conversión int() lanza ValueError cuando el texto no representa un entero.
5–7	except ValueError	Camino del error: informa y devuelve None.
8–10	else	Camino del éxito: solo se ejecuta si no hubo excepción; confirma la conversión y devuelve el valor.
11–12	finally	Se ejecuta en ambos caminos, incluso con el return de por medio: cierra el intento de conversión.
15–16	dos llamadas de prueba	Con "42" se observa try → else → finally; con "hola" se observa try → except → finally.

Experimento en clase: quite el bloque except y observe el traceback completo; vuelva a ponerlo y compare la salida.

Código 3 · cliente_tcp_robusto.py

```python
import socket


def main():
```

```
    try:
        conexion = socket.create_connection(("127.0.0.1", 5000), timeout=5)
    except ConnectionRefusedError:
        print("El servidor no está disponible (conexión rechazada)")
        return
    except TimeoutError:
        print("El servidor no respondió a tiempo")
        return

    with conexion:
        try:
            conexion.sendall("hola\n".encode("utf-8"))
            respuesta = conexion.recv(1024).decode("utf-8").strip()
            print(f"Servidor responde: {respuesta}")
        except ConnectionResetError:
            print("El servidor cerró la conexión inesperadamente")


main()
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
5–6	create_connection con timeout	Intenta conectar al puerto 5000; timeout=5 evita esperar indefinidamente.
7–9	except ConnectionRefusedError	No hay servidor escuchando: se informa y el cliente termina de forma limpia con return.
10–12	except TimeoutError	El servidor no respondió a tiempo: mensaje claro y salida ordenada.
14	with conexion	Pase lo que pase dentro, el socket se cierra al salir del bloque.
15–18	envío y recepción vigilados	El intercambio con framing por \n va dentro de un segundo try.
19–20	except ConnectionResetError	El servidor cerró de golpe a media conversación: se informa y with cierra el socket.

Código 4 · servidor_tcp_robusto.py

```
import socket
import threading


def atender_cliente(conexion, direccion):
    """Atiende a un cliente sin que sus errores detengan al servidor."""
    try:
        with conexion:
            buffer = ""
            while True:
                datos = conexion.recv(1024)
                if not datos:
                    break
                buffer += datos.decode("utf-8")
```

```
15                 while "\n" in buffer:
16                     linea, buffer = buffer.split("\n", 1)
17                     conexion.sendall((linea + "\n").encode("utf-8"))
18     except ConnectionResetError:
19         print(f"El cliente {direccion} se desconectó abruptamente")
20     except BrokenPipeError:
21         print(f"No se pudo enviar al cliente {direccion}")
22     finally:
23         print(f"Conexión cerrada: {direccion}")
24
25
26 def main():
27     servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
28     servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
29     try:
30         servidor.bind(("0.0.0.0", 5000))
31     except OSError:
32         print("El puerto 5000 está ocupado por otro proceso")
33         return
34     servidor.listen()
35     print("Servidor TCP escuchando en el puerto 5000")
36     while True:
37         conexion, direccion = servidor.accept()
38         hilo = threading.Thread(target=atender_cliente,
39                                 args=(conexion, direccion),
40                                 daemon=True)
41         hilo.start()
42
43
44 main()
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
5–6	def atender_cliente	Misma función de la guía anterior, ahora envuelta en manejo de errores.
7–17	try con el ciclo de eco	Todo el trabajo del hilo (with, recv, framing, sendall) queda vigilado.
18–19	except ConnectionResetError	El cliente se fue de golpe: solo muere este hilo; el servidor sigue aceptando clientes.
20–21	except BrokenPipeError	No se pudo enviar porque la conexión ya murió: se registra y el hilo termina.
22–23	finally	El cierre se registra siempre, tanto en desconexión limpia como en error.
29–33	bind vigilado	Si el puerto 5000 está ocupado, OSError lo captura y el servidor informa en lugar de tronar.

Línea(s)	Elemento	Explicación
36–41	accept y un hilo por conexión	El ciclo principal no cambia: acepta y delega; los errores viven dentro de cada hilo.

Prueba en clase: conecte un cliente y ciérrelo con Ctrl+C a media conversación; el servidor debe seguir atendiendo a los demás.

Código 5 · cliente_udp_reintentos.py

```
1  import socket
2  import time
3
4
5  def consultar(servidor, mensaje, reintentos=3):
6      """Envía una consulta UDP y reintenta si no hay respuesta."""
7      with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
8          cliente.settimeout(2)
9          for intento in range(1, reintentos + 1):
10             try:
11                 cliente.sendto((mensaje + "\n").encode("utf-8"), servidor)
12                 datos, _ = cliente.recvfrom(1024)
13                 return datos.decode("utf-8").strip()
14             except TimeoutError:
15                 print(f"Intento {intento}: sin respuesta")
16                 time.sleep(intento)
17     return None
18
19
20 respuesta = consultar(("127.0.0.1", 5001), "ESTADO")
21 print(respuesta if respuesta else "El servidor no respondió")
```

Explicación línea por línea

Línea(s)	Elemento	Explicación
5–6	def consultar(servidor, mensaje, reintentos=3)	Función reutilizable: recibe la dirección del servidor, el mensaje y el máximo de intentos.
7–8	socket UDP y settimeout(2)	Cada recvfrom() espera a lo más 2 segundos; en UDP el timeout es la única señal de pérdida.
9	for intento in range(1, reintentos + 1)	Ciclo de reintentos acotado: nunca un while True sin salida.
10–13	try: enviar y esperar	Envía el datagrama y espera la respuesta; si llega, la devuelve de inmediato con return.
14–16	except TimeoutError	Sin respuesta: informa el número de intento y espera un tiempo creciente (intento segundos) antes de reintentar.
17	return None	Si se agotaron los intentos, devuelve None para que el llamador decida qué hacer.

Línea(s)	Elemento	Explicación
20–21	uso de la función	El llamador interpreta el None con un mensaje claro: el servidor no respondió.

4. Ejercicios de consolidación

Ejercicios progresivos que extienden la guía anterior (hilos, TCP y UDP) con el manejo de errores de esta guía. Se indican los requisitos y las pistas de qué elementos usar; **la solución se construye en clase**.

Ejercicio 1 — Hilos con nombres propios

- **Enunciado:** a partir del hola mundo con hilos, crear tres hilos cuyos nombres y número de saludos salgan de una lista de tuplas, por ejemplo [("Ana", 3), ("Beto", 5), ("Cora", 2)], y verificar que todos terminan antes del mensaje final.
- **Pistas:** un ciclo for que crea los Thread guardándolos en una lista; un segundo ciclo para hacer join() a cada uno.
- **Se evalúa:** salida entrelazada, mensaje final siempre al final, código limpio.

Ejercicio 2 — Servidor de eco con contador por cliente

- **Enunciado:** extender el servidor de eco de la guía anterior para que cada respuesta lleve el número de mensajes que ese cliente ha enviado: el cliente manda hola\n y recibe 1>hola\n, luego 2>otro\n, y así sucesivamente.
- **Pistas:** el contador es una variable local de atender_cliente (un entero que se incrementa por línea recibida); se construye la respuesta con un f-string.
- **Se evalúa:** el conteo es independiente por cliente; dos clientes simultáneos no comparten contador.

Ejercicio 3 — Cliente que nunca falla

- **Enunciado:** escribir un cliente TCP que pida al usuario la IP del servidor y maneje todas las salidas de fallo: servidor apagado, IP inválida, respuesta que nunca llega y desconexión a media conversación. En cada caso debe informar con un mensaje claro y terminar con código de salida distinto de cero.
- **Pistas:** Código 3 como base; socket.gaierror aparece cuando la IP o el nombre no se pueden resolver; sys.exit(1) termina con código de error.
- **Se evalúa:** ninguna excepción llega al usuario como traceback; cada fallo tiene su mensaje propio.

Ejercicio 4 — Consulta UDP con reintentos y espera creciente

- **Enunciado:** aplicar la función consultar() del Código 5 al protocolo de LA FILA: enviar CUANTOS al servidor y, si no responde, reintentar hasta tres veces duplicando la espera en cada intento (1, 2 y 4 segundos).
- **Pistas:** la espera sale de time.sleep(2 ** (intento - 1)); el None final debe traducirse en un mensaje y no en un error.
- **Se evalúa:** con el servidor apagado se observan los tres intentos con sus pausas; encendiéndolo a medio ciclo, la consulta tiene éxito.

Ejercicio 5 — LA FILA tolerante a fallos (extensión de la Práctica 6)

- **Enunciado:** extender el servidor de turnos de la Práctica 6 para que ningún cliente pueda tumbarlo: desconexiones abruptas, mensajes que no siguen el protocolo (algo distinto de TURNO>apodo) y bytes que no son UTF-8 válido deben quedar registrados en consola y el servidor debe seguir asignando turnos sin repetir números.
- **Pistas:** Código 4 como plantilla del hilo de atención; validar el mensaje con startswith("TURNO>") antes de procesarlo; el contador compartido sigue protegido con threading.Lock.
- **Se evalúa:** demostración en clase con un cliente que envía basura a propósito y dos clientes legítimos que reciben turnos consecutivos.

Ejercicio 6 — Registro de errores en archivo (desafío)

- **Enunciado:** agregar al servidor del Ejercicio 5 un registro (log) en un archivo de texto: cada error registrado escribe una línea con fecha, hora, dirección del cliente y descripción, por ejemplo 2026-10-06 10:32:15 (187.141.12.9, 51020) conexión reiniciada por el cliente.
- **Pistas:** time.strftime("%Y-%m-%d %H:%M:%S") da la marca de tiempo; el archivo se abre en modo "a" (agregar) dentro de cada except, o se mantiene abierto protegido con el mismo Lock del contador.
- **Se evalúa:** el archivo crece con cada error provocado en la demostración y ninguna línea queda incompleta aunque dos hilos registren al mismo tiempo.

5. Errores típicos y cómo evitarlos

Contexto	Error	Cómo evitarlo
Manejo	except: sin tipo de excepción	Atrapa todo y esconde errores de programación; capturar siempre el tipo específico.
Manejo	except que solo dice pass	El error desaparece sin dejar rastro; todo except debe informar o registrar.
Manejo	except OSError antes que sus subclases	La subclase jamás se alcanza; ordenar de lo específico a lo general.

Contexto	Error	Cómo evitarlo
Manejo	Meter todo el programa en un solo try	Imposible saber qué línea falló; un try por cada operación que puede fallar.
Servidor	try / except fuera del hilo de atención	El fallo de un cliente tira al servidor completo; el manejo va dentro de atender_cliente.
UDP	Reintentar con while True sin límite	Si el servidor no existe, el ciclo no termina nunca; reintentos acotados con for.
UDP	No usar settimeout en el cliente	recvfrom() espera para siempre ante un datagrama perdido; siempre con timeout.
Ambos	Olvidar cerrar el socket tras un error	Se agotan los descriptores del proceso; with o finally garantizan el cierre.